

Базові структури алгоритмів та їх представлення

1. Поняття алгоритму

Алгоритм — це набір кроків, який описує спосіб розв’язання певного завдання.

Поняття походить від імені математика Аль-Хорезмі, який описав позиційну десяткову систему числення.

Чітке формальне визначення алгоритму сформувалося лише у XX столітті.

2. Властивості алгоритму

1. Скінченність

Алгоритм повинен завершуватися після виконання певної кількості кроків.

2. Дискретність

Процес складається з окремих кроків, кожен з яких виконується незалежно.

3. Визначеність

Кожен крок алгоритму повинен бути чітким і однозначним.

4. Наявність вхідних даних

Алгоритм працює з початковими даними.

5. Наявність вихідних даних

Після завершення роботи алгоритм повинен повертати результат.

6. Масовість

Алгоритм застосовується для цілого класу однотипних задач.

7. Формальність

Алгоритм не залежить від зовнішніх факторів; його може виконувати будь-який виконавець.

8. Результативність

Для всіх допустимих вхідних даних алгоритм повинен або дати результат, або визначити, що задача нерозв’язна.

9. Зрозумілість

Виконавець повинен чітко розуміти, які операції потрібно виконувати на кожному кроці.

3. Способи представлення алгоритмів

Алгоритми можуть бути записані у різних формах:

1. Словесний опис (вербальний)

Алгоритм описується звичайною мовою.

2. Псевдокод

Формальний запис, що дозволяє описувати алгоритми незалежно від конкретної мови програмування.

3. Схемне представлення (графічне)

Використовуються структурограми або блок-схеми.

4. Графічне представлення алгоритмів

Алгоритми можуть бути показані за допомогою блок-схем, діаграм діяльності, діаграм станів.

Блок-схема — найпоширеніший графічний спосіб опису алгоритму.

5. Блок-схеми: основи

Для блок-схем використовують стандартизовані графічні символи згідно з ISO 5807 (ДСТУ ISO 5807:2016).

Кожен символ означає певну дію або тип операції:

- початок і кінець алгоритму
- операції
- введення/виведення
- перевірку умови
- з'єднання та переходи

Символи повинні бути приблизно одного розміру, рівномірно розташованими, текст у них має бути коротким та інформативним.

6. Правила застосування символів блок-схем

1. Кожен символ має певне значення і використовується незалежно від тексту всередині.
2. Символи розташовують рівномірно, без зайвих перетинів ліній.
3. Якщо текст не поміщається в символ, використовують окремий блок-коментар.
4. Потоки даних і керування позначаються лініями. Стандартний напрямок — зліва направо і згори вниз.

5. Якщо напрямок інший — обов'язково ставляться стрілки.
6. У блоках входи повинні бути зліва або зверху, виходи — справа або знизу.
7. Уникають перетину ліній; за потреби лінії розривають.

7. Види блок-схем алгоритмів

1. Лінійні алгоритми

Дії виконуються послідовно, одна за одною.

2. Алгоритми з розгалуженням

Виконання залежить від умови; передбачено два або більше варіантів подальших дій.

3. Циклічні алгоритми

Одні й ті самі дії повторюються багато разів.

Це найпоширеніший тип алгоритмів у задачах перебору і чисельних методах.

8. Ефективність алгоритмів

Ефективність визначається тим, наскільки швидко алгоритм працює або скільки пам'яті він використовує.

Теорія складності обчислень вивчає часову та просторову складність алгоритмів.

Аналіз алгоритмів складається з визначення залежності кількості операцій від розміру вхідних даних.

Алгоритм вважається ефективним, якщо кількість кроків зростає повільно при збільшенні розміру задачі.

9. Асимптотична складність алгоритмів

При великому обсязі даних оцінюють поведінку алгоритму за допомогою нотації "O" (O-символ, нотація Ландау).

Нехай $f(N)$ — функція, що визначає кількість операцій алгоритму при обробці N елементів.

Якщо існує константа K така, що

$$|f(N)| \leq K \cdot |g(N)|,$$

то пишуть:

$$f(N) = O(g(N))$$

Це означає, що $g(N)$ асимптотично оцінює верхню межу складності алгоритму.

Приклад

Якщо алгоритм виконує N разів цикл із 4 операцій, а поза циклом ще 4 операції, то:

$$f(N) = 4N + 4$$

Після спрощення отримуємо:

$$f(N) = O(N)$$

Це лінійна складність.

10. Коректність алгоритму

Алгоритм повинен працювати правильно для всіх дозволених вхідних даних.

Виділяють два види коректності:

1. Часткова коректність

Якщо алгоритм завершився, то результат правильний.

2. Повна коректність

Алгоритм завжди завершується і видає правильний результат.

Для доведення коректності використовують:

- передумову (P)
- алгоритм (A)
- післяумову (R)

Це записують у вигляді:

$$P \{A\} R$$

Передумова має бути істинною перед запуском, післяумова — після завершення алгоритму.

Формальні методи коректності застосовуються у складних системах, включно з автоматикою, критичними системами, ІТ-безпекою, верифікацією апаратних і програмних схем.